Αναγνώριση Προτύπων - Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 13, 2025

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Κατόπιν, υλοποιούμε τη συνάρτηση *parse_free_digits* προκειμένου να διαβάσουμε σωστά τα αρχεία με τη βιβλιοθήκη librosa.

```
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Το αποτέλεσμα της κλήσης της συνάρτησης είναι:

Total wavs: 3000. Fs = 8000 Hz

Στην συνέχεια βασισμένοι στην υπολοποίηση του βοηθητικού κώδικα ορίζουμε τη δικιά μας συνάρτηση *split_free_digits* την οποία παρουσιάζουμε παρακάτω:

```
def extract_features(wavs, n_mfcc=6, Fs=8000):
    # Extract MFCCs for all wavs
    window = 30 * Fs // 1000
    step = window // 2
    frames = [
        librosa.feature.mfcc(
            y=wav, sr=Fs, n_fft=window, hop_length=window - step, n_mfcc=n_mfcc
        ).T
        for wav in tqdm(wavs, desc="Extracting mfcc features...")
```

```
    ]

    print("Feature extraction completed with {} mfccs per frame".format(n_mfcc)
    )

    return frames


def split_free_digits(frames, ids, speakers, labels):
    print("Splitting in train test split using the default dataset split")
    # Split to train-test
    X_train, y_train, spk_train = [], [], []
    X_test, y_test, spk_test = [], [], []
    test_indices = ["0", "1", "2", "3", "4"]

    for idx, frame, label, spk in zip(ids, frames, labels, speakers):
        if str(idx) in test_indices:
            X_test.append(frame)
            y_test.append(label)
            spk_test.append(spk)
        else:
            X_train.append(frame)
            y_train.append(label)
            spk_train.append(spk)

    return X_train, X_test, y_train, y_test, spk_train, spk_test
```

Τρέχουμε τον κώδικα για να επιβεβαιώσουμε την ορθή λειτουργία του:

```
└─ python3 lab.py
Total wavs: 3000. Fs = 8000 Hz
Splitting in train test split using the default dataset split
Train dataset's size is 2400
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 9
Train dataset has 60 samples of digit 9
```

Βλέπουμε ότι το split που έχουμε κάνει έχει μέγεθος 2400, δηλαδή είναι πράγματι το 80% του 3000. Επιπλέον, είναι ισορροπημένο, αφού υπάρχει ίδιος αριθμός δειγμάτων ήχου σε κάθε κατηγορία - ψηφίο (60). Επομένως ως μετρική του *accuracy* θα είναι αρκετό για να αξιολογήσουμε την επίδοση των ταξινομητών μας (όπως είδαμε και στην προπαρασκευή, στην περίπτωση του ισορροπημένου dataset το F1-score ακολουθεί το ίδιο trend με το accuracy).

Αφού επιβεβαιώσαμε την ορθή λειτουργία της ρουτίνας μας αφαιρούμε - μετατρέπουμε σε σχόλιο τον κώδικα για το debugging.

Βήμα 10

Βήμα 11

Βήμα 12

Βήμα 13

Βήμα 14

1

Η υλοποίηση του βήματος αυτού βασίζεται στη δομή του βοηθητικού κώδικα για το εργαστήριο και συγκεκριμένα στο αρχείο *lstm.py*. Έχουν συμπληρωθεί τα τμήματα σημασμένα ως TODO και επιπλέον έχουν τροποποιηθεί λειτουργικότητες και προστεθεί επιπλεόν συναρτήσεις ώστε να καλυφθούν τα υποερωτήματα του βήματος.

2

Για την κατασκευή του LSTM δικτύου αξιοποιήσαμε τη βιβλιοθήκη PyTorch. Ο κώδικας φαίνεται παρακάτω:

```
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Κατά την αρχικοποίηση του δικτύου ελέγχεται κατά πόσο η αρχιτεκτονική του είναι αμφίδρομη ή όχι. Εφόσον είναι, αφενός τίθεται σε True η αντίστοιχη παράμετρος και αφετέρου φροντίζουμε ώστε να διπλασιαστεί το μέγεθος του hidden state του δικτύου.

Στη συνέχεια, επισημαίνουμε τα είδη των layers που θα χρησιμοποιηθούν στην υλοποίηση του δικτύου. Το κυρίως layer είναι το nn.LSTM, το οποίο ρυθμίζεται με τις παραμέτρους του με τρόπο τέτοιο ώστε να καλύψει τις απαιτήσεις της εκφώνησης στα πιθανά ερωτήματα που θα προκύψουν. Το dropout layer δεν χρησιμοποιείται πάντα. Αυτό το καθορίζουμε με την παράμετρο dropout του nn.Dropout layer. Εάν αυτή είναι 0 προφανώς αυτή η τεχνική regularization δεν χρησιμοποιείται. Επιπλέον, αξίζει να αναφέρουμε ότι έχουμε φροντίσει το Dropout να μην εφαρμοστεί στο τελευταίο layer. Το nn.Linear layer χρησιμοποιείται ώστε να πάρει τα output του LSTM (για αυτό έχει και μέγεθος όσο του hidden state) και να τα συνδυάσει προκειμένου να πάρει την τελική απόφαση για

το classification του ψηφίου.

Η συνάρτηση forward είναι αυτή που καθορίζει το πώς γίνεται το inference του μοντέλου. Δηλαδή, κατ' επέκταση, τη διάταξη των δομικών layer που ορίσαμε και εξηγήσαμε παραπάνω. Ο πρώτος διαχωρισμός που κάνουμε έγκειται στο αν χρησιμοποιούμε την τεχνική packing στο input (χρειάζεται για το ερώτημα 8 - Bonus). Με την τεχνική του padding πετυχαίνουμε 2 πράγματα. Αρχικά, σε dataset με δεδομένα ακολουθίες διαφορετικών μηκών, είναι σκόπινο να ενώσουμε τις μικρότερες από αυτές ώστε να τις επεξεργαζόμαστε ταυτόχρονα και να εξοικονομούμε χρόνο κατά την εκπαίδευση. Επιπρόσθετα, αποφεύγουμε το zero-padding και την επεξεργασία αυτής της παραμορφωμένης εκδοχής των ακολουθιών εισόδου. Αυτή η τεχνική είναι ιδιαίτερα χρήσιμη σε δίκτυα στα οποία το μέγεθος εισόδου είναι αυστηρά καθορισμένο, όπως οι Transformers. Ο τρόπος που επιλέγουμε να εκτελέσουμε το packing είναι αρχικά να ταξινομήσουμε τις ακολουθίες με τα φωνήματα κατά φθίνον μήκος έτσι ώστε να τις συνδυάσουμε αποτελεσματικότερα. Αυτός ο συνδυασμός πραγματοποιείται από την συνάρτηση *packed_padded_sequence()* της βιβλιοθήκης της Pytorch (βλέπουμε ότι το enforced_sorted είναι ορισμένο σε True).

Τέλος, όπως βλέπει κανείς εδώ:

```
        # TODO: YOUR CODE HERE
        # Check if the metric has improved
        if self.best is None:
            return True
```

```
        elif self.mode == "max":
            return value > self.best
        else:
            raise ValueError(f"Invalid mode '{self.mode}'. Use 'min' or 'max'."
    )


class FrameLevelDataset(Dataset):
    def __init__(self, feats, labels):
        """
        feats: Python list of numpy arrays that contain the sequence features.
               Each element of this list is a numpy array of shape seq_length x
     feature_dimension
        labels: Python list that contains the label for each sequence (each
    label must be an integer)
        """
        # TODO: YOUR CODE HERE
        self.lengths = [len(i) for i in feats]
```

Έτσι εξασφαλίζουμε ότι το dropout δε θα μπει στο τελευταίο layer.

3

Σε αυτό το βήμα καλούμαστε να εκπαιδεύσουμε το μοντέλο. Έτσι, αρχικά, θα εξηγήσουμε τη διαδικασία της εκπαίδευσής του.

```
        self.feature_size = rnn_size * 2 if self.bidirectional else rnn_size

        # TODO: YOUR CODE HERE
        # ---------------- Insert your code here ---------------- #
        # Initialize the LSTM, Dropout, Output layers
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=rnn_size,
            num_layers=num_layers,
            batch_first=True,
```

```
            dropout=dropout if num_layers > 1 else 0.0,
            bidirectional=bidirectional,
        )

        self.dropout = nn.Dropout(dropout)
        self.output_layer = nn.Linear(self.feature_size, output_dim)


    def forward(self, x, lengths):
        """
        x : 3D numpy array of dimension N x L x D
            N: batch index
            L: sequence index
            D: feature index

        lengths: N x 1
        """
```

Στο trainning loop παίρνουμε τα δεδομένα του dataloader batch by batch, υπολογίζουμε το loss και κάνουμε το ανάλογο βήμα στον optimizer.

Ο optimizer που χρησιμοποιούμε είναι ο AdamW. Αξιοποιεί όρους ορμής και έχει μοντέλο για να μεταβάλει τους όρους του learning rate και της ταχύτητας κατάλληλα ώστε να επιτύχει την καλύτερη σύγλιση. Προφανώς εκτυπώνουμε το loss σε κάθε εποχή όπως μας ζητείται στην εκφώνηση.

4

Στην συνάρτηση train:

```
def train(models, optimizers, train_dataloader, val_dataloader, criterion):
    # TODO: YOUR CODE HERE
    metrics = {
        "training_loss": {},
        "validation_loss": {},
        "validation_accuracy": {},
    }

    confusion_matrices = {}

    best_model_name, best_val_acc = None, -1

    for model_name, model, optimizer in zip(models.keys(), models.values(),
    optimizers.values()):
        model.to(device)

        early_stopping = EarlyStopping(patience, mode="min", base_model=None)

        print(f"Training in {device}")

        metrics["training_loss"][model_name] = []
        metrics["validation_loss"][model_name] = []
        metrics["validation_accuracy"][model_name] = []
        for epoch in range(epochs):
            training_loss = training_loop(model, train_dataloader, optimizer,
    criterion)
            valid_loss, y_pred, y_true = evaluation_loop(model, val_dataloader,
     criterion)

            # TODO: Calculate and print accuracy score
            valid_accuracy = accuracy_score(y_true, y_pred)
            print(
                "Epoch {}: train loss = {}, valid loss = {}, valid acc = {}".
    format(
```

```
                    epoch, training_loss, valid_loss, valid_accuracy
                )
            )
            metrics["training_loss"][model_name].append(training_loss)
            metrics["validation_loss"][model_name].append(valid_loss)
            metrics["validation_accuracy"][model_name].append(valid_accuracy)

            if valid_accuracy > best_val_acc:
                best_model_name = model_name

            if early_stopping.stop(valid_loss, model):
                print("early stopping...")
                model_weights = torch.load(checkpoint_filename, weights_only=
    False)
                break

    return best_model_name, metrics, models
```

Παίρνουμε τις μετρικές που μας ενδιαφέρουν, δηλαδή το trainning και validation loss καθώς και το validation accuracy. Η master train είναι αυτή που εκπαιδεύει όλα τα μοντέλα που μας ενδιαφέρουν.

5

Καλούμαστε να προσθέσουμε L2 Regularization στο μοντέλο μας, καθώς και dropout.

Το L2 regularization δεν είναι κάτι το οποίο επηρεάζει δομικά το μοντέλο μας άμεσα. Είναι ένας όρος ποινικοποίησης του μέτρου των βαρών που προστίθεται στο loss function. Έτσι, μειώνουμε την πολυπλοκότητα του μοντέλου και έχουμε ως αποτέλεσμα την αποφυγή του overfitting. Αυτό είναι σημαντικό και αναμένεται να παίξει ρόλο, ειδικά σε ένα απλό dataset όπως είναι το δικό μας.

- epochs: 750
- learning rate: $2 * 10^{-}5$
- dropout: 0.4